

Casamento De Padrões

João Eduardo Montandon
DCC024

Uma função é...

$$F = D \rightarrow I$$

Se toda função mapeia um domínio para uma imagem... Como diferenciar cada uma? 🤔

Padrão: uma expressão que pode ser mapeada para outras expressões

```
- fun f n = n * n;  
val f = fn : int -> int  
- fun f (x, y) = x * y;  
val f = fn : int * int -> int
```

- `n` : mapeia qualquer valor para um argumento
 - `(x, y)` : mapeia uma tupla para os componentes `a` e `b`
-

Exemplos

1. Underscore (`_`)
 2. Constantes
 3. Tuplas
 4. Listas
-

Underscore (`_`)

```
- fun hello _ = "oi";  
val hello = fn : 'a -> string  
- hello 2;
```

```
val it = "oi" : string
- hello [];
val it = "oi" : string
```

Constantes

```
- fun hello 0 = "oi";
Warning: match nonexhaustive
         0 => ...
val hello = fn : int -> string
- hello 0;
val it = "oi" : string
```

Non-Exhaustive Match

- Alguns padrões apresentaram o aviso "match non-exhaustive", o que significa?
- Padrão não cobre todos os possíveis valores para aquele tipo

```
- hello 1;
uncaught exception Match [nonexhaustive match failure]
  raised at: stdIn:94.19
```

Tuplas

```
- val par = (1, 2);
val par = (1,2) : int * int
- val (x, y) = par;
val x = 1 : int
val y = 2 : int
```

Você já viu procedimento parecido antes em ML? 🤔

Listas De Padrões

```
- val [h, _] = [1,2];
Warning: binding not exhaustive
```

```
val h = 1 : int
```

- Casa qualquer lista de tamanho 2
- Trata `h` e `_` como sub-padrões, ligando `h` ao primeiro elemento

Podem ser aplicados a operação de construção da lista (`::`)

```
- fun first (um::resto) = um;  
Warning: match nonexhaustive  
val first = fn : 'a list -> 'a  
- first([1,2,3,4,5,6])  
val it = 1 : int
```

Como poderia pegar o 2o elemento da lista? 🤔

Padrões Conhecidos

1. Variável ➡ casa qualquer coisa e se liga a variável
2. `_` ➡ casa qualquer coisa
3. Constante ➡ casa apenas aquela constante
4. Tupla ➡ casa uma tupla cujo elementos se casam com sub-padrões
5. Lista ➡ casa lista de qualquer tamanho cujo elementos se casam com sub-padrões
6. `::` ➡ casa qualquer lista não-vazia `tail`

Funções

- ML permite que especifiquemos diferentes padrões para os parâmetros de uma determinada função
- Função será executada conforme o casamento realizado

```
- fun f 0 = "zero"  
= | f 1 = "um";  
Warning: match nonexhaustive  
val f = fn : int -> string  
- f 0;  
val it = "zero" : string
```

```
- f 1;  
val it = "um" : string
```

Padrões Sobrepostos

Padrões podem se sobrepor...

```
fun f 0 = "zero"  
| fun _ = "outro";  
  
f 0; (* 0 que será retornado? *)  
f 34; (* E aqui? *)
```

ML irá executar a função que se encaixar primeiro

Estilo Pattern-Matching

Qual é mais legível? 🤖

```
fun f 0 = "zero"  
| fun _ = "outro";
```

```
fun f 0 =  
if n = 0 then "zero"  
else "outro";
```

-
- Em ML, dê preferência a declarações que utilizam casamento de padrões
 - Geralmente são mais curtas e legíveis
-

Exemplo: Fatorial

```
fun fact n =  
if n = 0 then 1 else n * fact(n-1)
```

Qual seria a implementação com padrões?

```
fun fact 0 = 0
| fact n = n * fact(n-1);
```

Exemplo: Lista Reversa

Sem estilo

```
fun reversa L = if null L then nil else reversa(tl L) @ [hd L];
```

Com estilo 🕶️

```
fun reversa nil = nil
| reversa (h::t) = reversa t @ [h];
```

Outros Exemplos

Qual seria a implementação com padrões para...

- Somatório?
- Contagem?
- Fibonacci?

Nem Tudo São Flores

Problema: crie uma função para verificar se dois valores são iguais

```
- fun f(a, a) = true
| f(a, b) = false;
Error: duplicate variable in pattern(s): a
```

Não é permitido utilizar a mesma variável mais de uma vez para o mesmo padrão!

O Comando `let`

```
<let-exp> ::= let <definicoes> in <expressao> end;
```

- Permite a definição de variáveis visíveis apenas em um determinado bloco
 - Definições podem ser utilizadas a partir do momento em que foram definidas
 - Permite quebrar longas expressões em partes menores
-

```
- let
    val x = 1
    val y = 2
in
    x + y
end;
val it = 3 : int
- x;
Error: unbound variable or constructor: x
```

Exemplo: Conversor De Tempo

```
fun days2secs days =
    let
        val hours = days * 24
        val minutes = hours * 60
    in
        minutes * 60
    end;
```

- Cálculo foi quebrado entre as diferentes variáveis
-

Exemplo: Merge Sort

01: Particionador

- Entrada: [1, 2, 3, 4, 5, 6, 7, 8]
- Saída: ([1, 3, 5, 7], [2, 4, 6, 8])

```
fun part nil = (nil, nil)
| part [a] = ([a], nil)
| part (a::b::resto) =
    let
        val (x, y) = part resto
    in
        (a :: x, b :: y)
    end;
```

Onde que isso é útil? 🤔

02: Juntar Listas Ordenadas

```
fun merge (nil, y) = y
| merge (x, nil) = x
| merge (xs, ys) =
    let
        val x = hd xs
        val y = hd ys
        val restoX = tl xs
        val restoY = tl ys
    in
        if (x < y) then x :: merge(restoX, ys)
        else y :: merge(xs, restoY)
    end;
```

Tem um jeito melhor de se fazer? 🤔

03: Ordenar

```
fun mergeSort nil = nil
| mergeSort [a] = [a]
| mergeSort L =
    let
        val (p1, p2) = part L
    in
        merge(mergeSort p1, mergeSort p2)
    end;
```

Funções Aninhadas

- Se funções podem ser interpretadas como variáveis, também podemos incluir suas definições localmente usando `let`
- Útil para funções auxiliares (*helper functions*)

```
fun mergeSort nil = nil
| mergeSort [a] = [a]
| mergeSort L =
    let
        (* Função para realizar partição *)
        fun part nil = (nil, nil)
          | part [a] = ([a], nil)
          | part (a::b::resto) =
              let
                  val (x, y) = part resto
              in
                  (a :: x, b :: y)
              end;

        (* Função para realizar merge *)
        fun merge (nil, y) = y
          | merge (x, nil) = x
          | merge (xs, ys) =
              let
                  val x = hd xs
                  val y = hd ys
                  val restoX = tl xs
                  val restoY = tl ys
              in
                  if (x < y) then x :: merge(restoX,
ys)
                  else y :: merge(xs, restoY)
              end;
        val (p1, p2) = part L
    in
        merge(mergeSort p1, mergeSort p2)
    end;
```

Conclusões

- Um padrão é uma expressão cuja sintaxe pode se encaixar em outra expressão do programa

- ML suporta diversos mecanismos de casamento de padrões
- Esses recursos aumentam consideravelmente a expressividade da linguagem